Contents

Exercise 1 — Your first honest workflow

Set: Friday 25 September 2026 · **Due:** Friday 2 October 2026, start of Lesson 2

Roughly 2–3 hours. Work alone; discussing ideas with others is fine, sharing notebooks is not.

Goal

Run the workflow from notebook 01 yourself on a dataset you have not seen, and — this is the part that carries the marks — **justify the decisions** rather than only making them. By the end you should be able to say what your number means and what it does not.

Dataset

The wine quality dataset: 4898 white wines described by 11 physico-chemical measurements, each with a quality score from 3 to 9 awarded by tasters.

```
from sklearn.datasets import fetch_openml

# OpenML ships the columns as V1...V11 and the scores renumbered 1-7. Put both back.
COLUMNS = ["fixed_acidity", "volatile_acidity", "citric_acid", "residual_sugar",
           "chlorides", "free_sulfur_dioxide", "total_sulfur_dioxide",
           "density", "pH", "sulphates", "alcohol"]

wine = fetch_openml("wine-quality-white", version=1, as_frame=True, parser="liac-arff")
X = wine.data.set_axis(COLUMNS, axis=1)
y = wine.target.astype(int) + 2        # back to the 3-9 the tasters awarded
```

Check that yourself before going further — `y.value_counts().sort_index()` should run from 3 to 9, and `X.describe()` should show an alcohol column averaging about 10.5% by volume. If your target runs 1 to 7 you have skipped the `+ 2`, and every threshold you choose in task 1 will be two points off what you meant.

`parser="liac-arff"` only pins the reader scikit-learn 1.3 uses by default, so the result does not change under a newer version; without it you get a `FutureWarning`.

The first call downloads and caches it; afterwards it works offline. If your network blocks it, the course image has it cached — say so at the start of Lesson 2 rather than struggling alone.

Two things to notice before you begin, because they shape everything:

- **Quality is an integer from 3 to 9**, so this is neither plain classification nor plain regression. Part of the exercise is deciding how to treat it, and defending the decision.
- **The classes are very unevenly distributed.** Most wines score 5 or 6. Check this first.

Tasks

1. Frame the problem (write, do not code)

In a markdown cell, answer in a few sentences each:

- What exactly are you predicting, and how did you decide to treat the quality score? If you turn it into a binary "good / not good" problem, state your threshold and justify it.
- Who would use such a model, and for what decision?
- Which error is worse, and why?

2. Look at the data

Size, types, missing values, and the distribution of the target. Produce **one figure** that shows the class imbalance clearly.

3. Split, before anything else

Hold out a test set. Justify your proportion and say why you did or did not stratify.

4. Baseline

Build the trivial model and report its score. Everything after this is measured against it.

5. Model

Fit a logistic regression inside a `Pipeline` with a `StandardScaler`.

State in one sentence **why the scaler must be inside the pipeline** rather than applied to the whole dataset beforehand. Marks are for the reason, not the fact.

6. Evaluate honestly

- Report accuracy, and then explain why it is inadequate here.

- Produce a confusion matrix and read it in words: which cases does the model get wrong, and does it fail symmetrically?
- Choose a metric appropriate to the error you called worse in task 1, and report it.

7. Measure your own uncertainty

Repeat the split with **at least 10 different random seeds** and report the minimum, maximum and mean of your metric.

Then answer: if you had run this once and reported a single number, how misleading could that number have been?

8. Conclude

In no more than 150 words: what did you find, how much do you trust it, and what would you do next with another week?

What to hand in

A single notebook named `{surname}_01.ipynb` that:

- runs top to bottom in the course container with no edits
- has `random_state` fixed everywhere, so your numbers reproduce
- keeps its outputs, so it can be read without being run
- alternates markdown and code — the prose is not decoration, it is where the reasoning lives

---

Assessment criteria

Criterion	Weight	What earns marks
Methodological correctness	40%	Split before anything is learnt; preprocessing inside the pipeline; a baseline that makes the comparison meaningful; no test-set peeking
Implementation	30%	Runs clean, reproducible, readable; figures that support the argument
Interpretation and communication	30%	Decisions justified rather than stated; the confusion matrix read in words; honest account of uncertainty in task 7

Note there are **no marks for accuracy**. A modest score reached by a sound route beats a better one reached by an unsound one — and any submission that scales the full dataset before splitting loses the methodology marks regardless of its result.

Hints

- Stuck on task 1? There is no single right answer. A defended choice earns full marks; an undefended one does not.
- `sklearn.dummy.DummyClassifier` gives you task 4 in two lines.
- For task 6, `classification_report` prints per-class precision and recall. Read the column for the class you said matters.
- Task 7 is a loop over `random_state` values. Do not use `cross_val_score` yet — doing it manually is what makes the variability visible, and we formalise it in Lesson 5.